

5.8.1 Introduzione alla Computazione Evolutiva

Un modo di sviluppare **algoritmi di ottimizzazione globali parallelizzabili** nel campo dell'intelligenza artificiale è la **computazione evolutiva** (evolutionary computation), che si ispira alla simulazione dell'evoluzione biologica.

5.8.1.1 Concetto Fondamentale

L'idea di base è quella di applicare alla soluzione di problemi informatico-ingegneristici lo stesso approccio che ha condotto, nel corso dell'evoluzione naturale, alla formazione di forme di vita sempre più adatte all'ambiente.

Nella computazione evolutiva le regole sono tipicamente quelle della **selezione naturale**, con variazioni dovute all'incrocio e/o alla mutazione. Il comportamento emergente desiderato è:

- Realizzazione di **soluzioni di alta qualità** per problemi complessi
- **Capacità di adattamento** a un ambiente che cambia

5.8.1.2 Evoluzione come Metodo di Ricerca

L'evoluzione biologica può essere vista come un **metodo di ricerca** all'interno di un grandissimo numero di possibili "soluzioni":

- **Spazio delle possibilità:** Insieme di tutte le sequenze genetiche
 - **Soluzioni desiderate:** Organismi altamente adatti (forte capacità di sopravvivere e riprodursi)
 - **Suggerimento:** Vantaggi dell'Approccio Evolutivo
 - **Parallelismo intrinseco:** L'evoluzione mette alla prova milioni di specie in parallelo
 - **Regole semplici:** Variazione casuale + selezione naturale
 - **Risultati complessi:** Straordinaria varietà e complessità nella biosfera
-

5.8.2 Cellular Automata

Il concetto di **Cellular Automata** fu introdotto da **Ulam e von Neumann** nel 1966. Si tratta di un algoritmo evolutivo che opera in modo **discreto nello spazio e nel tempo**.

5.8.2.1 Definizione

Nell'algoritmo viene definita una **popolazione p di cellule**. In applicazioni di image processing, le cellule corrispondono ai **pixel** (o voxel) di un'immagine.

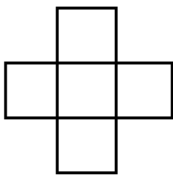
Ogni cellula è definita dalla tripletta:

$$A = (S, N, \delta)$$

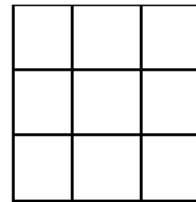
Componente	Descrizione
S	Stato della cellula (indice del pattern a cui appartiene)
N	Kernel centrato sulla cellula per il calcolo dello stato successivo
δ	Regola di transizione dallo stato corrente t allo stato $t + 1$

5.8.2.2 Tipi di Kernel

Esempi tipici di kernel sono:



Kenel di von Newmann



Kernel di Moore

Figura 1: Pasted image 20251201190412.png

Il kernel è definito dalla sua **dimensione** (tipicamente dispari) e **forma**.

5.8.2.3 Procedura

1. **Definizione stato iniziale:** $S(0)$ per $t = 0$
2. **Calcolo nuovo stato:** Per tutta la popolazione p , calcolare $S(1)$ per $t + 1$
3. **Verifica convergenza:** Se soddisfatta ci si ferma, altrimenti calcolare S all'istante successivo

5.8.3 Applicazione alla Segmentazione

5.8.3.1 Modello per Immagini Biomediche

Consideriamo un'immagine biomedica C a livelli di grigio di dimensione $N \times N$. Avremo $N \times N$ cellule, ognuna corrispondente a un pixel.

Lo **stato S** per ogni pixel p è definito dalla tripletta:

$$(I_p, T_p, C_p)$$

Elemento	Significato
I_p	Intero 1..K che definisce la classe di appartenenza
T_p	"Forza" della cellula p
C_p	Livello di grigio del pixel p

Il risultato è il valore di I che rappresenta la **segmentazione** dell'immagine in K pattern (analogamente al k-means).

5.8.3.2 Esempio: Immagine Binaria (K=2)

Inizializzazione:

- $I = 0$ ovunque
- $T = 0$ ovunque
- $I(a, b) = 1$ e $T(a, b) = 1$ (seed nel punto a,b)

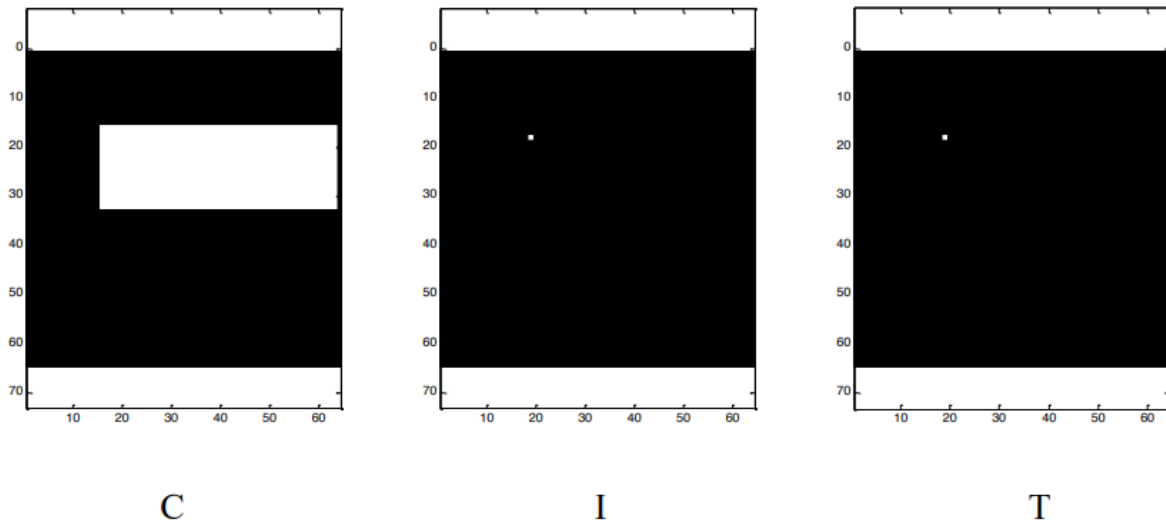


Figura 2: Pasted image 20251201190445.png

5.8.3.3 Regola di Transizione

La regola δ è basata sull'idea che se una cellula è "**più forte**" della vicina (T più alto), la ingloba copiandoci dentro il suo stato.

La **forza** di una cellula p rispetto alla sua vicina q è definita da:

$$F(p) = T(p) \left(1 - \frac{\|C_p - C_q\|}{\max \|C\|} \right)$$

Se $F(p) > T(q)$, lo stato di p viene copiato in q .

□ **Info:** Interpretazione

La forza di p è uguale al suo valore di T moltiplicato per un fattore $[0,1]$:

- **= 1** se i pixel hanno lo stesso livello di grigio
- **= 0** se la differenza è uguale alla dinamica dell'immagine

Questo pone una “**barriera**” all’espansione quando c’è alto gradiente.

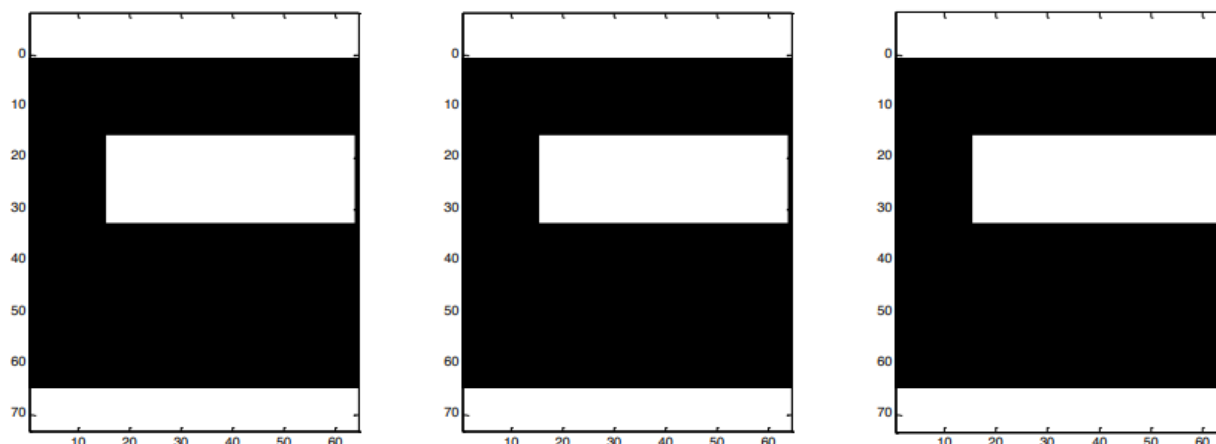


Figura 3: Pasted image 20251201190517.png

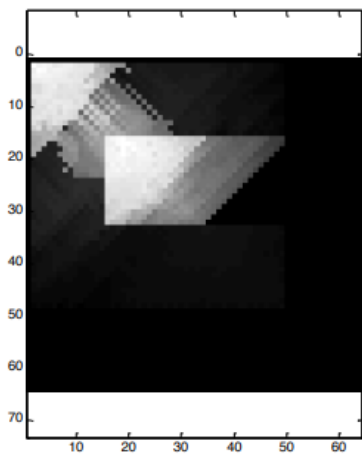
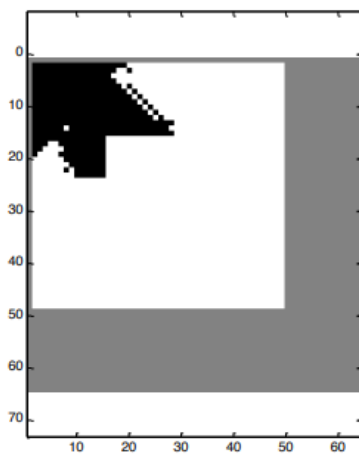
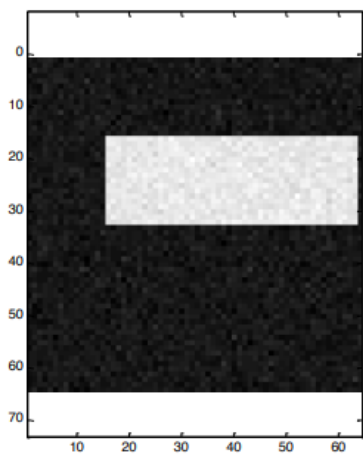
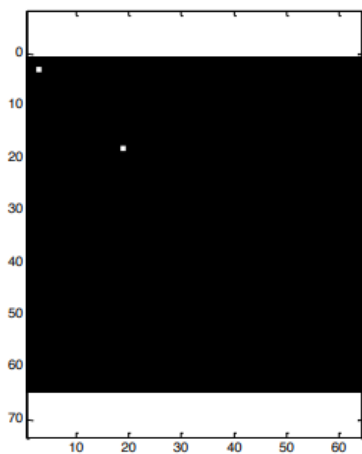
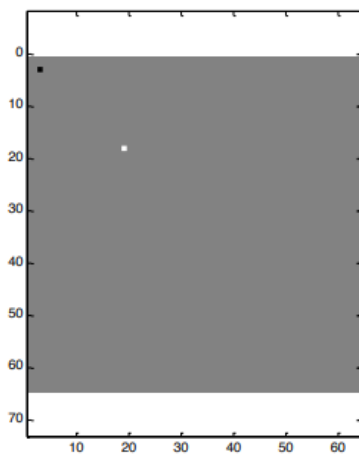
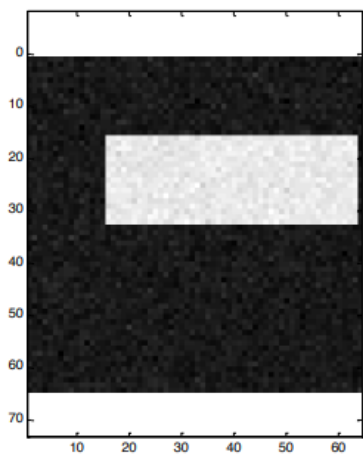
Il risultato è analogo a un **algoritmo di labeling**: il seed si espande inglobando le cellule fino alla barriera di transizione.

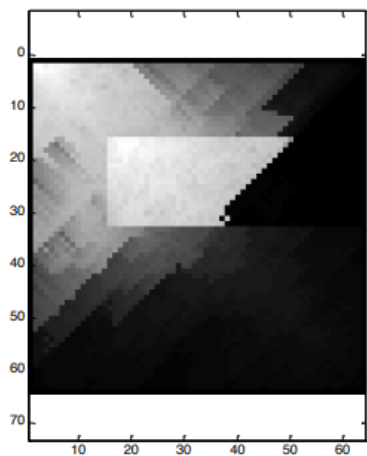
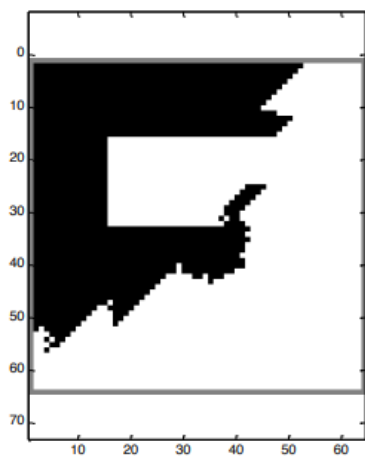
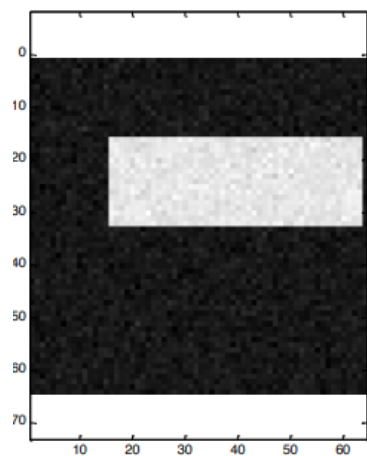
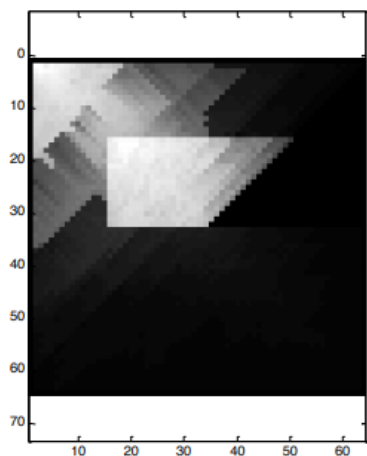
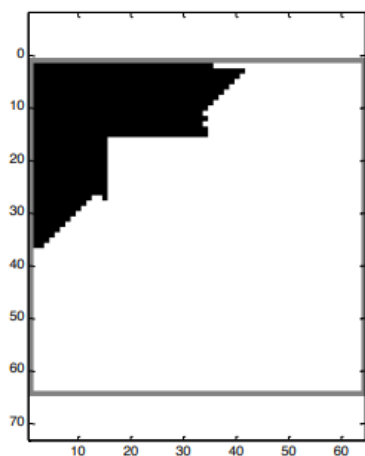
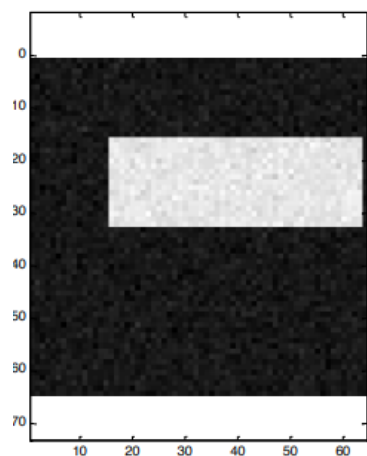
5.8.4 Segmentazione con Rumore

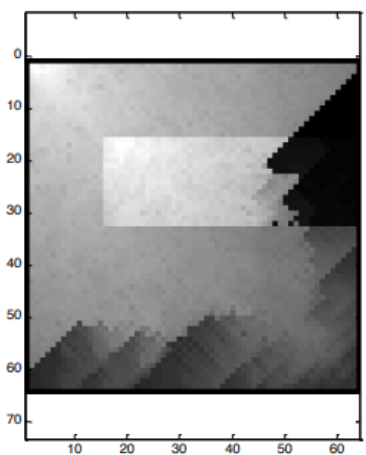
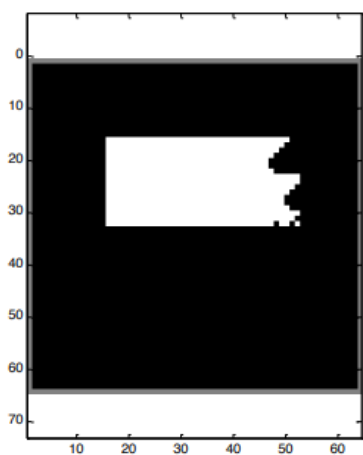
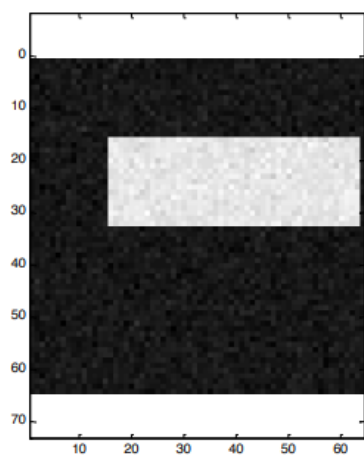
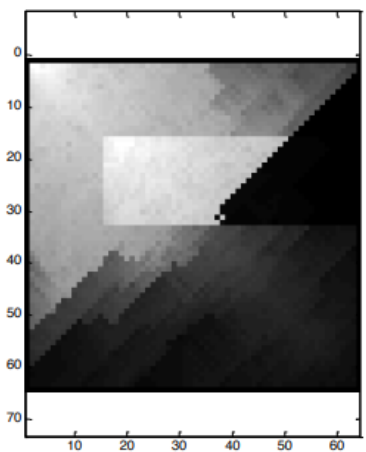
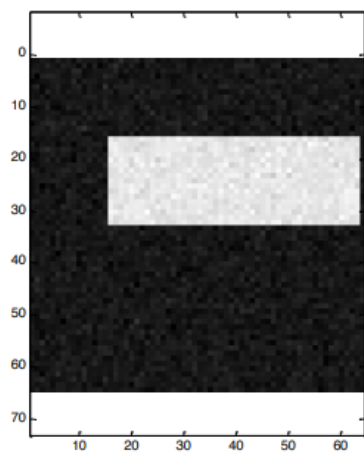
Per immagini corrotte da rumore, è opportuno definire **K=3** classi:

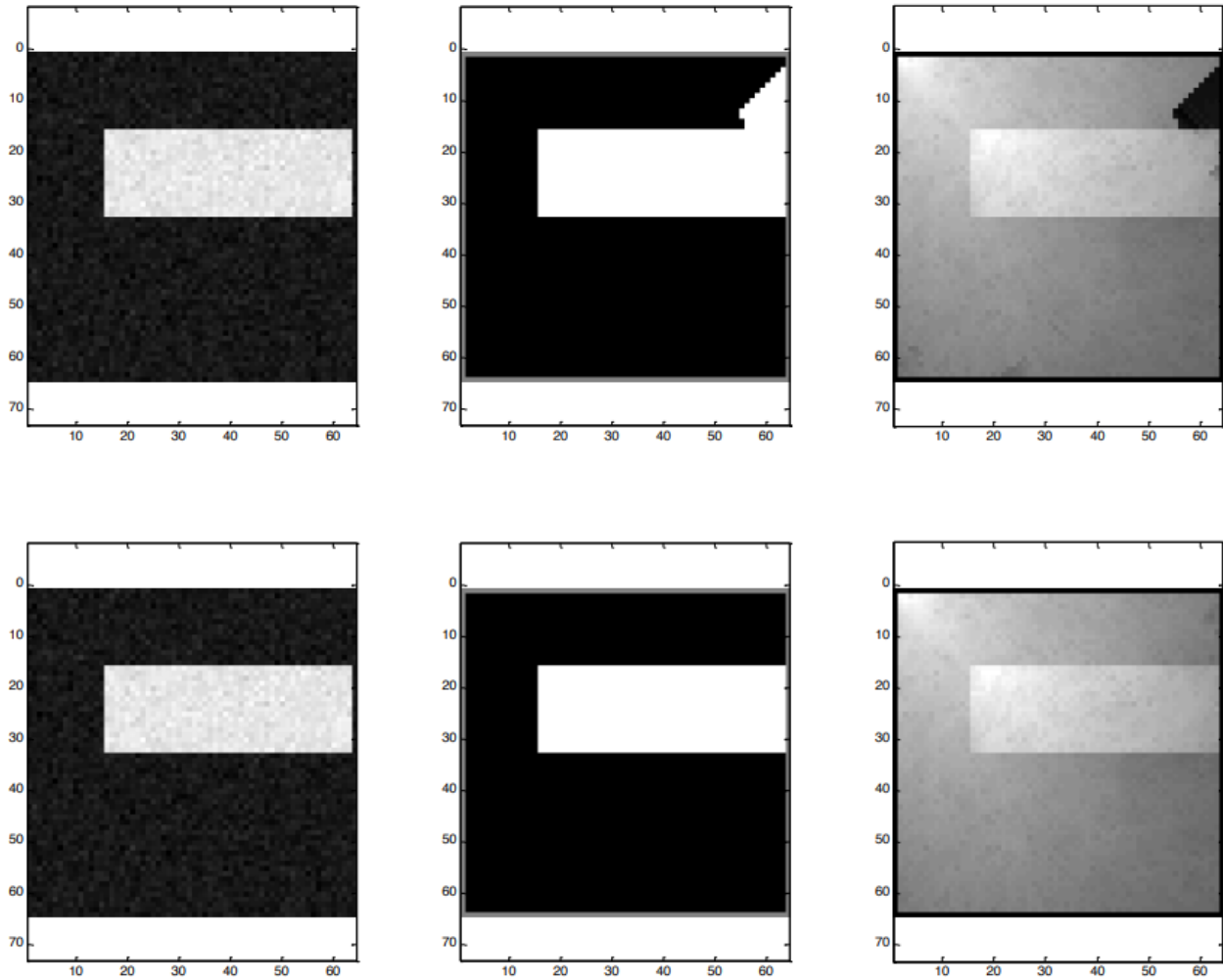
- **2:** Oggetto
- **0:** Sfondo
- **1:** Pixel non definiti

Si parte da **due seed**: uno sull’oggetto e uno sullo sfondo.









Le due popolazioni (oggetto e sfondo) **competono** tra loro fino ad eliminare la terza popolazione, ottenendo una segmentazione analoga a quella di un algoritmo di **region growing**.